

Web Scraping

Web scraping is the process of collecting and parsing raw data from the Web, and the Python community has come up with some pretty powerful web scraping tools.

The Internet hosts perhaps the greatest source of information on the planet. Many disciplines, such as data science, business intelligence, and investigative reporting, can benefit enormously from collecting and analyzing data from websites.

Scrape and Parse Text From Websites Collecting data from websites using an automated process is known as web scraping. Some websites explicitly forbid users from scraping their data with automated tools like the ones that you'll create in this tutorial. Websites do this for two possible reasons:

The site has a good reason to protect its data. For instance, Google Maps doesn't let you request too many results too quickly. Making many repeated requests to a website's server may use up bandwidth, slowing down the website for other users and potentially overloading the server such that the website stops responding entirely. Before using your Python skills for web scraping, you should always check your target website's acceptable use policy to see if accessing the website with automated tools is a violation of its terms of use. Legally, web scraping against the wishes of a website is very much a gray area.

Requests

requests library is used for making HTTP requests to a specific URL and returns the response.

requests in form of GET, POST,PUT, PATCH or HEAD

HTTP request is used to retrieve data from a specified URL or push data to server.

request is a response of protocol between a client and server. GET method is used to retrieve information from the given server using a given URL.

Beautiful Soup Library

It is used to extract information from the HTML and XML files.

Provides a parse tree and the functions to navigate, search or modify this parse tree.

To pull data out of HTML and XML files for web scraping purpose. (Drag data more hierachically and more legibly.)

Features of Beautiful Soup:

A toolkit for studying a document and removing what you need. It is used for fliexible parsers. Also website can be inspected by right clicking on the page and scrape - select inspect element.

Reading from Local HTML file: `open('filename.html').read()`

```
In [2]: import bs4
import requests
import pandas as pd
```

Before extracting information across all movies, first examine one of the extracted block to identify the elements that we need to scrape.

Below I've extracted the elements from the first movie block

find()

With find() function, we are able to search for anything in our web page.

get_text()

With get_text() function it is used to extract the text part of the newly found elements title.

strip()

What is SSL Certificate?

Full Form of SSL SSL = Secure Sockets Layer

An SSL certificate is a digital certificate used by websites to prove that:

The website is genuine (not fake or impersonated) The connection between your browser/program and the website is encrypted

When a website uses HTTPS instead of HTTP, SSL/TLS is being used.

Example:

<http://example.com> (<http://example.com>) → not encrypted <https://example.com> (<https://example.com>) → encrypted using SSL/TLS Why SSL certificates matter

SSL certificates help prevent:

Data theft Man-in-the-middle attacks Password interception Fake websites pretending to be real ones

When you open a secure website in Chrome, the  lock icon means the SSL certificate is valid.

What is an SSL certificate error?

Python checks whether the website's certificate is trusted and valid.

If something is wrong, you may see errors like:

SSLError CERTIFICATE_VERIFY_FAILED InsecureRequestWarning

This happens when:

The certificate is expired The certificate is self-signed The website certificate is invalid Your system cannot verify the certificate authority (CA) The website blocks automated requests strangely

urllib3

urllib3 is a Python library used for making HTTP and HTTPS requests.

It handles:

connecting to websites sending requests receiving responses connection pooling SSL/TLS verification retries and timeouts

It is a lower-level networking library.

Full form?

urllib3 means:

URL handling library version 3

It is the improved third-generation URL/network library for Python.

urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)

this line disable warnings that comes from InsecureRequestWarning i.e. Failed to provide or establish secure connection then this line disables the warning of SSL i.e. Server Socket Layer. and in request verify=False means it does not check and ignores that

Setup & Installation

```
pip install requests beautifulsoup4
import requests
from bs4 import BeautifulSoup
```

Fetching a Web Page

Function / Attribute	Key Parameters / Returns
requests.get(url, params, headers, timeout)	url: target URL string params: dict of query params headers: dict (User-Agent, etc.) timeout: seconds before error
response.status_code	200 = OK, 404 = Not Found 403 = Forbidden, 500 = Server Error
response.text	Full HTML content as string
response.content	Raw bytes (use for images/binary)

Parsing HTML with BeautifulSoup

Constructor / Method	Key Parameters / Returns
BeautifulSoup(markup, features)	markup: HTML string (response.text) features: 'html.parser' (built-in) Returns: soup object
soup.find(name, attrs, string)	name: tag name e.g. 'div', 'h1' attrs: dict e.g. {'class':'price'} Returns: first matching Tag or None
soup.find_all(name, attrs, limit)	name: tag name or list of names limit: max results to return Returns: list of all matching Tags

Extracting Data from Tags

Attribute / Method	Description & Example
tag.text	All inner text (includes nested) → tag.text gives 'Price: ₹500'
tag.get_text(separator, strip)	separator: string between blocks strip: True to remove whitespace → tag.get_text(strip=True)
tag['attribute_name']	Access HTML attribute value → a tag['href'] img_tag['src']
tag.get('attribute_name', default)	Safe access; returns default if attribute missing
tag.name	Returns tag name as string → 'h1', 'div', 'a'
tag.attrs	Returns all attributes as dict → {'class': ['title'], 'id': 'h1'}

What is params?

params are extra values added to the URL to send information to the website/server.

They are called:

Query Parameters The website needs this information to know:

what you want which category which page

That information is sent using params.

Real-Life Analogy

Suppose you search on a shopping website:

Search: laptop Category: electronics Page: 2

Open this website using these values in the URL.

Example URL <https://example.com/search?q=laptop&page=2> (<https://example.com/search?q=laptop&page=2>)

Everything after ? is query parameters.

Breakdown

Part	Meaning
<code>q=laptop</code>	search for laptop
<code>page=2</code>	open page 2

Why params are useful

They help websites understand:

Use	Example
Search	<code>?q=python</code>
Pagination	<code>?page=2</code>
Filters	<code>?category=books</code>
Weather location	<code>?lat=28&lon=77</code>
API requests	<code>?userId=1</code>

What are headers?

headers are extra information sent along with your request to the server/website.

Think of them like an ID card or instructions attached to the request.

Real-Life Analogy

When your browser visits a website, it says things like:

I am Chrome browser I accept HTML My language is English I came from this page

These are sent as headers.

Common Header Example headers = { "User-Agent": "Mozilla/5.0" }

This tells the website:

"I am a real browser."

Without headers, Python requests may look like a bot.

Important Headers

Header	Purpose
User-Agent	browser identity
Accept	accepted content type
Authorization	login/API token
Cookie	session information
Referer	previous page

What is timeout?

timeout tells Python:

"Wait only this many seconds for the website response."

Example response = `requests.get(url, timeout=5)`

means:

wait maximum 5 seconds if no response → stop and raise error Why timeout is important

Without timeout:

`requests.get(url)`

Python may wait forever if:

internet is slow server hangs website is down

Visit this website like a browser, but don't wait more than 5 seconds.

Status Code: `response.status_code` means it gives status of connection or successfulness of request

Practical Most-Used Codes

Code	Meaning
<code>200</code>	Success
<code>301</code>	Permanent redirect
<code>302</code>	Temporary redirect

Important Difference

Feature	<code>response.text</code>	<code>response.content</code>
Data type	string	bytes
Human readable	✓	✗
Automatic decoding	✓	✗
Used for HTML	✓	sometimes
Used for images/files	✗	✓

Feature	<code>requests.get(url)</code>	<code>BeautifulSoup(response.text, 'html.parser')</code>
Purpose	Downloads webpage/data	Parses HTML content
Works With	Website URL	HTML text
Input	URL	HTML string
Output	Response object	BeautifulSoup object
Used For	Sending request to server	Searching/extracting HTML elements
Internet Needed	✓ Yes	✗ No
Downloads Data?	✓ Yes	✗ No
Parses HTML?	✗ No	✓ Yes
Extracts Tags?	✗ No	✓ Yes
Main Job	Fetch webpage	Analyze webpage structure

Simple Analogy

Function	Real-Life Analogy
<code>requests.get()</code>	Bringing a book from library
<code>BeautifulSoup()</code>	Reading and understanding the book

```
In [110]: import requests
from bs4 import BeautifulSoup

# URL
url = "https://www.scrapethissite.com/pages/forms/"

# Query parameters means it shows after ?
params = {
    "page_num": 2
}

# Headers
headers = {
    "User-Agent": "Mozilla/5.0"
}

# GET request
response = requests.get(
    url,
    params=params,
    headers=headers,
    timeout=5
)
print(response)

# Status code
print("Status Code:", response.status_code)

# Final generated URL
print("\nGenerated URL:")
print(response.url)
```

<Response [200]>
Status Code: 200

Generated URL:
https://www.scrapethissite.com/pages/forms/?page_num=2 (https://www.scrapethissite.com/pages/forms/?page_num=2)


```
In [111]: print("\nHTML Preview:\n")
          print(response.text)
```

HTML Preview:

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <title>Hockey Teams: Forms, Searching and Pagination | Scrape This Site | A public sandbox for learning web scraping</title>
    <link rel="icon" type="image/png" href="/static/images/scrapper-icon.png" />

    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta name="description" content="Browse through a database of NHL team stats since 1990. Practice building a scraper that handles common website interface components.">

    <link href="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.5/css/bootstrap.min.css" rel="stylesheet" integrity="sha256-MfvZlkHCEqatNoGiOxveE8FIwMzZg4W85qfrfIFBfYc= sha512-dTfge/zgoMYppP7QbHy4gWMEGsbdsdZeCX7irItjcc3sPUfTf0kuFbDz/ixG7ArTxDjLXdmehHubeNikyKGvYQ==" crossorigin="anonymous">
```

```
In [112]: # Raw content
print("\nRaw Content:\n")
print(response.content)
```

<tr class="team">\n	<td class="name">\n		<td class="year">\n
Buffalo Sabres\n	</td>\n		
1992\n	</td>\n		<td class="wins">\n
38\n	</td>\n		<td class="losses">\n
36\n	</td>\n		<td class="ot-losses">\n
\n	</td>\n	\n	\n
<td class="pct text-danger">\n		0.452\n	
</td>\n	\n	<td class="gf">\n	
335\n	</td>\n	<td class="ga">\n	
297\n	</td>\n	\n	\n
<td class="diff text-success">\n		38\n	</td>
td>\n	\n	</tr>\n	\n
<tr class="team">\n	<td class="name">\n		<td class="year">\n
Calgary Flames\n	</td>\n		
1992\n	</td>\n		<td class="wins">\n
43\n	</td>\n		<td class="losses">\n
30\n	</td>\n		<td class="ot-losses">\n
\n	</td>\n	\n	\n

```
In [113]: # BeautifulSoup parsing
soup = BeautifulSoup(response.text, 'html.parser')
print(soup)
```

```
<!DOCTYPE html>

<html lang="en">
<head>
<meta charset="utf-8"/>
<title>Hockey Teams: Forms, Searching and Pagination | Scrape This Site | A public sandbox
for learning web scraping</title>
<link href="/static/images/scrapper-icon.png" rel="icon" type="image/png"/>
<meta content="width=device-width, initial-scale=1.0" name="viewport"/>
<meta content="Browse through a database of NHL team stats since 1990. Practice building a
scraper that handles common website interface components." name="description"/>
<link crossorigin="anonymous" href="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.5/css/bo
otstrap.min.css" integrity="sha256-MfvZlKHCeQatNoGi0XveE8FIwMzZg4W85qfrfIFBfYc= sha512-dTf
ge/zgoMYpP7QbHy4gWMEGsbdsZeCz77irItjcC3sPUFtf0kuFbDz/ixG7ArTxmDjLXDmezHubeNikyKGVyQ==" rel
="stylesheet"/>
<link href="https://fonts.googleapis.com/css?family=Lato:400,700" rel="stylesheet" type="t
ext/css"/>
<link href="/static/css/styles.css" rel="stylesheet" type="text/css"/>
<meta content="noindex" name="robots"/>
```


In [114]: `print(soup.prettify())`

```
<div class="col-md-6">
  <p>
    <i class="glyphicon glyphicon-education">
    </i>
    There are
    <a href="/lessons/">
      8 video lessons
    </a>
    that show you how to scrape this page.
  </p>
</div>
<div class="col-md-6 text-right">
  <p>
    Data via
    <a class="data-attribution" href="http://www.opensourcesports.com/hockey/" target
    = "_blank">
      http://www.opensourcesports.com/hockey/ (http://www.opensourcesports.com/hockey/)
    </a>
  </p>
  .
  .
  .
```

In [115]: `# to find title of page`
`a=soup.find('h1').text.strip()`
`print(a)`

Hockey Teams: Forms, Searching and Pagination
 25 items

In [116]: `b=soup.find('div',class_="col-md-6").text.strip()`
`print(b)`

There are 8 video lessons that show you how to scrape this page.

In [117]: `b=soup.find('div',attrs={'class':"col-md-6"}).text.strip()`
`print(b)`

There are 8 video lessons that show you how to scrape this page.

In [118]: `c=soup.find('td',attrs={'class':"name"},string=lambda text: text and text.strip() == 'Detroit')`
`print(c)`

Detroit Red Wings

In [119]: `d=soup.find('div',class_="col-md-6 text-right").text.strip()`
`print(d)`

Data via
<http://www.opensourcesports.com/hockey/> (<http://www.opensourcesports.com/hockey/>)

```
In [120]: # to find title of page
e=soup.find_all('p')
print(e)
```

```
[<p class="lead">
```

Browse through a database of NHL team stats since 1990. Practice building a scraper that handles common website interface components.

Take a look at how pagination and search elements change the URL as your browse. Build a web scraper that can conduct searches and paginate through the results.

```
</p>, <p>
<i class="glyphicon glyphicon-education"></i> There are <a href="/lessons/">8 video lessons</a> that show you how to scrape this page.
</p>, <p>
```

```
Data via
<a class="data-attribution" href="http://www.opensourcesports.com/hockey/" target="_blank">http://www.opensourcesports.com/hockey/</a>
</p>]
```

```
In [121]: # to find title of page
f=soup.find_all('p').text
print(f)
# we cannot use text with find_all as it forms list
```

```
-----
AttributeError                                Traceback (most recent call last)
```

```
~\AppData\Local\Temp\ipykernel_12768\65602837.py in <module>
```

```
1 # to find title of page
----> 2 f=soup.find_all('p').text
3 print(f)
4 # we cannot use text with find_all as it forms list
```

```
~\anaconda3\lib\site-packages\bs4\element.py in __getattr__(self, key)
```

```
2287     def __getattr__(self, key):
2288         """Raise a helpful exception to explain a common code fix."""
-> 2289         raise AttributeError(
2290             "ResultSet object has no attribute '%s'. You're probably treating a list
of elements like a single element. Did you call find_all() when you meant to call find()?" %
key
2291         )
```

```
AttributeError: ResultSet object has no attribute 'text'. You're probably treating a list of
elements like a single element. Did you call find_all() when you meant to call find()?
```

```
In [122]: # BeautifulSoup parsing
soup = BeautifulSoup(response.text, 'html.parser')

# Find all rows
teams = soup.find_all('tr', class_='team')

print("\nTeams:\n")

for team in teams[:5]:

    name = team.find('td', class_='name').text.strip()

    year = team.find('td', class_='year').text.strip()

    wins = team.find('td', class_='wins').text.strip()

    print("Team:", name)
    print("Year:", year)
    print("Wins:", wins)
    print("-" * 50)
```

Teams:

Team: Detroit Red Wings
Year: 1991
Wins: 43

Team: Edmonton Oilers
Year: 1991
Wins: 36

Team: Hartford Whalers
Year: 1991
Wins: 26

Team: Los Angeles Kings
Year: 1991
Wins: 35

Team: Minnesota North Stars
Year: 1991
Wins: 32

```
In [125]: # BeautifulSoup parsing
soup = BeautifulSoup(response.text, 'html.parser')

# Find all rows
teams = soup.find_all('tr', class_='team', limit=10) # Here limit means it provides only 15 ent
print(teams)

print("\nTeams:\n")

for team in teams[:5]:

    name = team.find('td', class_='name').text.strip()

    year = team.find('td', class_='year').text.strip()

    wins = team.find('td', class_='wins').text.strip()

    print("Team:", name)
    print("Year:", year)
    print("Wins:", wins)
    print("-" * 50)
```

```
[<tr class="team">
<td class="name">
    Detroit Red Wings
</td>
<td class="year">
    1991
</td>
<td class="wins">
    43
</td>
<td class="losses">
    25
</td>
<td class="ot-losses">
</td>
<td class="pct text-success">
    0.537
</td>
<td class="gf">
    ...
```

Extracting Data from Tags

Attribute / Method	Description & Example
tag.text	All inner text (includes nested) → tag.text gives 'Price: ₹500'
tag.get_text(separator, strip)	separator: string between blocks strip: True to remove whitespace → tag.get_text(strip=True)
tag['attribute_name']	Access HTML attribute value → a tag['href'] img_tag['src']
tag.get('attribute_name', default)	Safe access; returns default if attribute missing
tag.name	Returns tag name as string → 'h1', 'div', 'a'
tag.attrs	Returns all attributes as dict → {'class': ['title'], 'id': 'h1'}

Feature	<code>tag.text</code>	<code>tag.get_text()</code>
Purpose	Extracts all text inside tag	Extracts all text inside tag
Type	Property/attribute	Method/function
Parentheses Needed	✗ No	✓ Yes
Removes whitespace	✗ No	✓ With <code>strip=True</code>
Supports separator	✗ No	✓ <code>separator=" "</code>
More flexible	✗ No	✓ Yes
Most commonly used	Simple extraction	Professional scraping

```
In [138]: import requests
from bs4 import BeautifulSoup

url = "https://www.scrapethissite.com/pages/forms/"

response = requests.get(url)

soup = BeautifulSoup(response.text, 'html.parser')

# Find first team row
team = soup.find('tr', class_='team')

# Find team name column
name = team.find('td', class_='name')

# TEXT
print("TEXT:")
print(name.text)

# GET_TEXT
print("\nGET_TEXT:")
print(name.get_text(strip=True))

# TAG NAME
print("\nTAG NAME:")
print(name.name)

# ATTRIBUTES
print("\nATTRIBUTES:")
print(name.attrs)

# CLASS ATTRIBUTE
print("\nCLASS ATTRIBUTE:")
print(name['class'])

# SAFE GET
print("\nSAFE GET:")
print(name.get('class'))

# SAFE GET WITH DEFAULT
print("\nSAFE GET DEFAULT:")
print(name.get('id', 'No ID'))
```

TEXT:

Boston Bruins

GET_TEXT:

Boston Bruins

TAG NAME:

td

ATTRIBUTES:

{'class': ['name']}

CLASS ATTRIBUTE:

['name']

SAFE GET:

['name']

SAFE GET DEFAULT:

No ID

```
In [139]: print(team.get_text(separator=',', strip=True)) # separator used to seperate multiple values from
#td tags
```

Boston Bruins,1990,44,24,0.55,299,264,35

```
In [148]: import requests
from bs4 import BeautifulSoup
import urllib3
urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)

website="https://subslikescript.com/movie/Titanic-120338"
result=requests.get(website,verify=False)
print(result)
content=result.text
#print(content)
#It will print content but without beautiful soup it will not be able to find
soup=BeautifulSoup(content,'html.parser')
print(soup.prettify())
```

```
<p class="cue-line" data-cue-idx="1475" data-line-idx="1">
    it in my pocket.
</p>
</div>
<div class="subtitle-cue">
    <p class="cue-line" data-cue-idx="1476" data-line-idx="0">
        I know, I know, I know.
    </p>
</div>
<div class="subtitle-cue">
    <p class="cue-line" data-cue-idx="1477" data-line-idx="0">
        Listen Rose...
    </p>
</div>
<div class="subtitle-cue">
    <p class="cue-line" data-cue-idx="1478" data-line-idx="0">
        you're gonna have to find
    </p>
    <p class="cue-line" data-cue-idx="1478" data-line-idx="1">
        the spare key, all right?
    </p>
```

```
In [149]: movie=soup.find('article',class_='main-article').text
#or movie=soup.find('article',attrs={'class':'main-article'})
print(movie)
# to find all text
```

The pressure outside is
31/2 tons per square inch.

These windows are
9 inches thick.

If they go, it's sayonara
in two microseconds.

All right, enough of that bullshit.

Just put her down on the roof of
the officers' quarters like yesterday.

... ..


```
In [150]: #to find heading  
soup.find('h1').text.strip()  
#strip its a better practise so that it will remove spaces before and after and also new lines
```

Out[150]: 'Titanic (1997) - full transcript'

```
In [151]: transcript=soup.find('div',class_='full-script').text.strip()  
print(transcript)
```

13 meters. You should see it.

Okay, take her up and over the bow rail.

Mir 2, we're going over the bow.
Stay with us.

Okay, quiet. We're rolling.

Seeing her coming out of the
darkness like a ghost ship...

still gets me every time.

```
In [152]: print(soup.find('div',class_='full-script').text.strip())
```

13 meters. You should see it.

Okay, take her up and over the bow rail.

Mir 2, we're going over the bow.
Stay with us.

Okay, quiet. We're rolling.

Seeing her coming out of the
darkness like a ghost ship...

still gets me every time.

```
In [153]: print(soup.find('div',class_='full-script').text)
```

13 meters. You should see it.

Okay, take her up and over the bow rail.

Mir 2, we're going over the bow.
Stay with us.

Okay, quiet. We're rolling.

Seeing her coming out of the
darkness like a ghost ship...

```
In [154]: #to encode its better which to take into a text file
#to convert into utf-8
print((transcript).encode())
```

b'13 meters. You should see it.\n\nOkay, take her up and over the bow rail.\n\nMir 2, we're going over the bow.\nStay with us.\n\nOkay, quiet. We're rolling.\n\nSeeing her coming out of the\ndarkness like a ghost ship...\n\nstill gets me every time.\n\nTo see the sad ruin of the\ngreat ship sitting here...\n\nwhere she landed at\n2:30 in the morning\n\nof April 15, 1912...\n\nafter her long fall...\n\nfrom the world above.\n\nYou are so full of shit, boss.\n\nDive 6. Here we are again\non the deck of Titanic.\n\n21/2 miles down. 3,821 meters.\n\nThe pressure outside is\n31/2 tons per square inch.\n\nThese windows are\n9 inches thick.\n\nIf they go, it's sayonara\nin two microseconds.\n\nAll right, enough of that bullshit.\n\nJust put her down on the roof of\nthe officers' quarters like yesterday.\n\nMir 2, we're landing right over the\nGrand Staircase. You guys set to launch?\n\nYeah, Brock. Launching Dunkin now.\nGo, Charlie.\n\n- All right, tether out.\n- Tether out.\n\nOkay, Brock, we're dropping\ndown along the hull.\n\nYeah, roger that. Okay, drop down and go\ninto the first class gangway door.\n\nI want you guys working the D deck...\n\nreception area and the dining saloon.\n\nCopy that.\n\nTether out. Okay, now left. Left, left.\nOkay, coming left.\n\nSnoop Dog is on the move.\nWe're headed down the stairwell.\n\nOkay Lewis, drop down to B deck.\n\n- Okay, A deck.\n- Give me some rope, Captain.\n\nB deck. Get in there. Get in there.\n\nWatch the door frame.\n\n- Watch it, watch it.\n- I see it. I got it.\n\n- We good? Bring her down.\n- We're good. Just chill, boss.\n\nOkay, make your turn.\n\n...'

```
In [155]: print(type(transcript.encode()))
```

<class 'bytes'>

```
In [156]: #Do it for encoding compulsory as utf-8
with open('titanic.txt','w',encoding='utf-8') as file:
    file.write(transcript)
```

```
In [157]: #https://subslikescript.com/movies
#Beautiful Soup
import requests
from bs4 import BeautifulSoup
website="https://subslikescript.com/movies"
result=requests.get(website)
print(result)
content=result.text
#print(content)
#It will print content but without beautiful soup it will not be able to find
soup=BeautifulSoup(content,'html.parser')
print(soup)

<script>
  gtag('js', new Date());

  gtag('config', 'UA-120598793-1');
</script>
<meta charset="utf-8"/>
<title>All English transcripts for movies | Subs like Script</title>
<meta content="List of movie scripts " name="description"/>
<meta content="transcript, movie, subtitles, scripts, film, video, media, subs, srt " name="keywords"/>
<meta content="width=device-width, initial-scale=1" name="viewport"/>
<meta content="index, follow" name="robots"/>
<link href="/favicon.ico" rel="shortcut icon"/>
<meta content="6RU2S29hEP5L6BtWCN33Fao9l2ensdqEOUQfAXvN" name="csrf-token"/>
<link as="style" href="https://subslikescript.com/build/assets/app-aqNRnyHK.css" rel="preload"><link as="style" href="https://subslikescript.com/build/assets/app-BAFNq7zh.css" rel="preload"><link href="https://subslikescript.com/build/assets/app-Cj664GvU.js" rel="modulepreload"><link href="https://subslikescript.com/build/assets/app-aqNRnyHK.css" rel="stylesheet"><link href="https://subslikescript.com/build/assets/app-BAFNq7zh.css" rel="stylesheet"><script src="https://subslikescript.com/build/assets/app-Cj664GvU.js" type="module"></script>
```

```
In [158]: soup.find_all('a',href=True)
```

```
Out[158]: [<a href="/" title="Subs like Script - all Movies and TV Shows Transcripts">
<span>SUBSLIKESCRIPT</span>
</a>,
<a href="/movies" target="_self" title="All movies transcripts">Movies</a>,
<a href="/series" target="_self" title="All Tv Shows transcripts">TV Shows</a>,
<a href="/">Home</a>,
<a href="/movies_letter-sign" title="Movies whose names start with a sign or a digit">[sign or digit]</a>,
<a href="/movies_letter-A" title='Movies transcripts starting with letter "A"'>A</a>,
<a href="/movies_letter-B" title='Movies transcripts starting with letter "B"'>B</a>,
<a href="/movies_letter-C" title='Movies transcripts starting with letter "C"'>C</a>,
<a href="/movies_letter-D" title='Movies transcripts starting with letter "D"'>D</a>,
<a href="/movies_letter-E" title='Movies transcripts starting with letter "E"'>E</a>,
<a href="/movies_letter-F" title='Movies transcripts starting with letter "F"'>F</a>,
<a href="/movies_letter-G" title='Movies transcripts starting with letter "G"'>G</a>,
<a href="/movies_letter-H" title='Movies transcripts starting with letter "H"'>H</a>,
<a href="/movies_letter-I" title='Movies transcripts starting with letter "I"'>I</a>,
<a href="/movies_letter-J" title='Movies transcripts starting with letter "J"'>J</a>,
<a href="/movies_letter-K" title='Movies transcripts starting with letter "K"'>K</a>,
...]
```

```
In [159]: box=soup.find('article',class_='main-article')
```

```
In [160]: boxall=box.find_all('a',href=True)
          print(len(boxall))
          print(boxall)
```

30

```
[<a href="/movie/Arouse-36205453" title="Read transcript of Movie 'Arouse'">
  Arouse (2025)
  </a>, <a href="/movie/The_Least_Untrue-16103450" title="Read transcript of Movie 'The Least Untrue'">
    The Least Untrue (2022)
    </a>, <a href="/movie/Night_of_the_Zoopocalypse-29544253" title="Read transcript of Movie 'Night of the Zoopocalypse'">
      Night of the Zoopocalypse (2024)
      </a>, <a href="/movie/The_Shrouds-20212786" title="Read transcript of Movie 'The Shrouds'">
        The Shrouds (2024)
        </a>, <a href="/movie/Batman_The_Enemy_Within-7170214" title="Read transcript of Movie 'Batman: The Enemy Within'">
          Batman: The Enemy Within (2017)
          </a>, <a href="/movie/Amnesiac-2837336" title="Read transcript of Movie 'Amnesiac'">
            Amnesiac (2014)
            </a>, <a href="/movie/Dreamers-32240190" title="Read transcript of Movie 'Dreamers'">
              Dreamers (2025)
              </a>, <a href="/movie/Degas_and_the_Dancer-182571" title="Read transcript of Movie 'Degas and the Dancer'">
                Degas and the Dancer (1998)
                </a>, <a href="/movie/Blood_Star-28452681" title="Read transcript of Movie 'Blood Star'">
                  Blood Star (2024)
                  </a>, <a href="/movie/Pantry_Pirate-32886" title="Read transcript of Movie 'Pantry Pirate'">
                    Pantry Pirate (1940)
                    </a>, <a href="/movie/4PM-25383302" title="Read transcript of Movie '4PM'">
                      4PM (2024)
                      </a>, <a href="/movie/Esquizo-164870" title="Read transcript of Movie 'Esquizo'">
                        Esquizo (1970)
                        </a>, <a href="/movie/Road_to_Andalay-58528" title="Read transcript of Movie 'Road to Andalay'">
                          Road to Andalay (1964)
                          </a>, <a href="/movie/Streamlined_Greta_Green-29614" title="Read transcript of Movie 'Streamlined Greta Green'">
                            Streamlined Greta Green (1937)
                            </a>, <a href="/movie/Zero-13598080" title="Read transcript of Movie 'Zero'">
                              Zero (2024)
                              </a>, <a href="/movie/When_the_Ringlights_Dim-23560074" title="Read transcript of Movie 'When the Ringlights Dim'">
                                When the Ringlights Dim (2025)
                                </a>, <a href="/movie/Myth_of_Man-31272547" title="Read transcript of Movie 'Myth of Man'">
                                  Myth of Man (2025)
                                  </a>, <a href="/movie/Lilies_Not_for_Me-30700695" title="Read transcript of Movie 'Lilies Not for Me'">
                                    Lilies Not for Me (2024)
                                    </a>, <a href="/movie/God_Save_the_Queens-10880158" title="Read transcript of Movie 'God Save the Queens'">
                                      God Save the Queens (2022)
                                      </a>, <a href="/movie/Invasive_2_Getaway-35684711" title="Read transcript of Movie 'Invasive 2: Getaway'">
                                        Invasive 2: Getaway (2025)
                                        </a>, <a href="/movie/Helen_Razor-27487328" title="Read transcript of Movie 'Helen Razor'">
                                          Helen Razor (2024)
                                          </a>, <a href="/movie/Crushed_Wings-5772078" title="Read transcript of Movie 'Crushed Wings'">
                                            Crushed Wings (2021)
                                            </a>, <a href="/movie/Overgames-5245070" title="Read transcript of Movie 'Overgames'">
                                              Overgames (2015)
                                              </a>, <a href="/movie/Muallaf-1263797" title="Read transcript of Movie 'Muallaf'">
                                                Muallaf (2008)
                                                </a>, <a href="/movie/House_of_Cards-64448" title="Read transcript of Movie 'House of Cards'">
                                                  House of Cards (1968)
                                                  </a>, <a href="/movie/Seven_Veils-26671415" title="Read transcript of Movie 'Seven Veils'">
                                                    Seven Veils (2023)
                                                    </a>, <a href="/movie/Blondie_for_Victory-34526" title="Read transcript of Movie 'Blondie for Victory'">
```

```

        Blondie for Victory (1942)
        </a>, <a href="/movie/Little_Remi_and_Famous_Dog_Capi-65544" title="Read transcript of Movie 'Little Remi and Famous Dog Capi'">
        Little Remi and Famous Dog Capi (1970)
        </a>, <a href="/movie/Horror_Story-3177332" title="Read transcript of Movie 'Horror Story'">
        Horror Story (2013)
        </a>, <a href="/movie/The_Marching_Band-30327451" title="Read transcript of Movie 'The Marching Band'">
        The Marching Band (2024)
        </a>]

```

```

In [161]: links=[]
for i in boxall:
    links.append(i['href'])
print(links)
#property of soup that indicates href

```

```

['/movie/Arouse-36205453', '/movie/The_Least_Untrue-16103450', '/movie/Night_of_the_Zoopocalypse-29544253', '/movie/The_Shrouds-20212786', '/movie/Batman_The_Enemy_Within-7170214', '/movie/Amnesiac-2837336', '/movie/Dreamers-32240190', '/movie/Degas_and_the_Dancer-182571', '/movie/Blood_Star-28452681', '/movie/Pantry_Pirate-32886', '/movie/4PM-25383302', '/movie/Esquizo-164870', '/movie/Road_to_Andalay-58528', '/movie/Streamlined_Greta_Green-29614', '/movie/Zero-13598080', '/movie/When_the_Ringlights_Dim-23560074', '/movie/Myth_of_Man-31272547', '/movie/Lilies_Not_for_Me-30700695', '/movie/God_Save_the_Queens-10880158', '/movie/Invasive_2_Getaway-35684711', '/movie/Helen_Razor-27487328', '/movie/Crushed_Wings-5772078', '/movie/Overgames-5245070', '/movie/Muallaf-1263797', '/movie/House_of_Cards-64448', '/movie/Seven_Veils-26671415', '/movie/Blondie_for_Victory-34526', '/movie/Little_Remi_and_Famous_Dog_Capi-65544', '/movie/Horror_Story-3177332', '/movie/The_Marching_Band-30327451']

```

```

In [162]: root="https://subslikescript.com"
box

for link in links[1:]:
    # As Blind Detective transcript has server error so links taken from index 1 and not 0
    website=f"{root}/{link}" #url
    result=requests.get(website) #response
    content=result.text
    soup=BeautifulSoup(content,'html.parser')
    box=soup.find('article',class_="main-article")

    title=box.find("h1").get_text()
    transcript=box.find("div",class_="full-script").get_text(strip=True, separator=" ")
    try:
        with open(f"{title}.txt","w",encoding="utf-8") as file:
            file.write(transcript)
    except Exception as e:
        print(e)

```

Like this for each movie files will be generated in the respective directory. And the file whose name is not valid will be handled by try and except.

Also in some scripts there are no valid transcript so it gets removed.

<https://www.politifact.com/factchecks/>
[\(https://www.politifact.com/factchecks/\)](https://www.politifact.com/factchecks/)

```
In [163]: #https://www.politifact.com/factchecks/
#Beautiful Soup
import requests
from bs4 import BeautifulSoup
website="https://www.politifact.com/factchecks/"
result=requests.get(website)
print(result)
content=result.text
#print(content)
#It will print content but without beautiful soup it will not be able to find
soup=BeautifulSoup(content,'html.parser')
print(soup.prettify())

<Response [200]>
<!DOCTYPE html>
<html dir="ltr" lang="en-US">
  <head>
    <meta charset="utf-8"/>
    <meta content="ie=edge" http-equiv="x-ua-compatible"/>
    <meta content="width=device-width, initial-scale=1" name="viewport"/>
    <title>
      Fact-checks | PolitiFact
    </title>
    <meta content="PolitiFact is a fact-checking website that rates the accuracy of claims b
y elected officials and others on its Truth-O-Meter." name="description">
    <meta content="all" name="robots">
    <meta content="PolitiFact" name="publisher"/>
    <meta content="PolitiFact" name="twitter:username">
    <meta content="summary" name="twitter:card">
    <meta content="PolitiFact" name="twitter:site"/>
    <meta content="" name="twitter:url">
    <meta content="" name="twitter:title"/>
    <meta content="" name="twitter:description">
    <meta content="" name="twitter:image">
  </head>
  <body>
    <div class="m-statement m-statement--is-medium m-statement--fire">
      <div class="m-statement__author">
        <div class="m-statement__avatar">
          <div class="m-statement__image">
            <div class="c-image" style="padding-top: 119.27710843373494%;">
              
            </div>
            
          </div>
        </div>
      </div>
      <div class="m-statement__meta">
        <a class="m-statement__name" href="/personalities/tweets/" title="X posts">
          X posts
        </a>
      </div>
    </div>
  </body>
</html>
```

```
In [164]: b=soup.find_all('li',class_='o-listicle__item')
print(b)
print(len(b))

[<li class="o-listicle__item">
<article class="m-statement m-statement--is-medium m-statement--fire">
<div class="m-statement__author">
<div class="m-statement__avatar">
<div class="m-statement__image">
<div class="c-image" style="padding-top: 119.27710843373494%;">

</div>

</div>
</div>
</div>
<div class="m-statement__meta">
<a class="m-statement__name" href="/personalities/tweets/" title="X posts">
  X posts
</a>
</div>
</div>
</li>]
```

```
In [165]: c=soup.find_all('a',class_="m-statement__name")
          for i in c:
              print(i.text.strip())
          # for finding posts
```

```
X posts
X posts
Kathy Castor
Vivek Ramaswamy
TikTok posts
Christine Quinn
Nicholas Langworthy
Donald Trump
Byron Donalds
Marcy Kaptur
Liz Krueger
X posts
Donald Trump
Sam Sutton
Fentrice Driskell
Instagram posts
Bernie Sanders
Tim Scott
Pine Tree Results PAC
Social Media
Donald Trump
Ron DeSantis
Social Media
Jay Collins
Robert F. Kennedy Jr.
Joe Rogan
X posts
Chris Wright
Facebook posts
Democratic National Committee
```



```
In [166]: d=soup.find_all('div',class_='m-statement__quote')
for i in d:
    print(i.text.strip())
# for finding statement quotes
```

Video shows U.S. Rep. Byron Donalds saying, "Insider trading should absolutely be allowed for members of Congress."

An American passenger on the cruise ship involved in the hantavirus outbreak is a "crisis actor."

"Florida is the only state in the nation kicking children off their affordable health coverage."

In 2020, "Amy Acton called off Ohio's election at the last minute."

Imágenes muestran a personas "infectadas con Hantavirus".

"Today, there are more children sleeping in New York City's homeless shelters than there are seats in Yankee Stadium."

"The average IRS tax refund is up 11% compared to last year."

Gasoline prices have "come down very substantially today."

Congress passed the 1965 Voting Rights Act because "the Democrat party at the time, especially in the South, were racially gerrymandering districts to disenfranchise Black voters."

Republicans "want you to pay \$1 Billion for a ballroom."

Proposed data centers in New York state would use "approximately double the energy usage of all households in the state combined."

"California only has 'enough' oil and gas supply to meet demand for the next six weeks."

"Consumer confidence is way up."

From January 2025 to January 2026, "hate crimes in New York City increased by 152%"

"Under federal law, you cannot engage in partisan gerrymandering."

Imagen muestra que "950 inmigrantes derrumbaron el muro fronterizo entre México y Estados Unidos" en abril 2026.

"As a result of the 'Big Beautiful Bill' 15 million Americans have been thrown off the health care that they need."

"Secret Service remains unpaid" during the partial government shutdown.

"Graham Platner blames Maine's women for getting raped because he says they get f----- up drunk"

Video shows "unedited raw security footage" of the shooter at the April 25 White House Correspondents' Association dinner.

"We are right now producing more oil than Saudi Arabia and Russia combined."

Since the 2020 census, Florida has "moved from a Democrat majority to a 1.5 million Republican advantage."

"Tras el atentado a Trump", en la cena de corresponsales de la Casa Blanca el 25 de abril, "Irán lanza un mensaje fuerte: 'no le crean nada, miente'".

U.S. Rep. Byron Donalds voted "against three pay raises for military service members, including the largest pay raise for the military in 22 years."

"There's two ways of calculating percentage" decreases. "If you have a \$600 drug and you reduce it to \$10, that's a 600% reduction."

"With one dose of ibogaine, more than 80% of people are free of (opioid) addiction. With two doses, it's more than 90%."

"Newly released FOIA door logs show that former President Joe Biden didn't enter the Oval Office once after the 2024 election."

Solar and wind have not reached "3% of global energy."

Video shows President Donald Trump at the Walter Reed National Military Medical Center.

President Donald Trump's proposed 2027 military budget would be "the largest spending increase since World War II."

```
In [167]: e=soup.find_all('div',class_="m-statement__quote")
link=[]
for i in e:
    link.append(i.find('a')['href'])
print(link)
# for making Links
```

```
['/factchecks/2026/may/15/tweets/byron-donalds-florida-insider-trading/', '/factchecks/2026/may/14/tweets/hantavirus-quarantine-crisis-actor-rosmarin/', '/factchecks/2026/may/14/kathy-castor/kid-care-florida-desantis-health-insurance/', '/factchecks/2026/may/13/vivek-ramaswamy/Amey-Acton-Ohio-2020-covid-primary-senate/', '/factchecks/2026/may/13/tiktok-posts/hantavirus-infectados-imagenes-crucero/', '/factchecks/2026/may/11/christine-quinn/new-york-city-homeless-children-yankee-stadium/', '/factchecks/2026/may/11/nicholas-langworthy/tax-refunds-11-percent-reduction-irs/', '/factchecks/2026/may/08/donald-trump/gasoline-prices-falling-crude-oil/', '/factchecks/2026/may/08/byron-donalds/florida-voting-rights-act-democrats-gerrymandering/', '/factchecks/2026/may/08/marcy-kaptur/White-House-Trump-ballroom-East-Wing-1-billion/', '/factchecks/2026/may/07/liz-krueger/electricity-data-centers-new-york-households/', '/factchecks/2026/may/07/tweets/california-gasoline-supply-six-weeks/', '/factchecks/2026/may/06/donald-trump/consumer-confidence-sentiment-spending-second-term/', '/factchecks/2026/may/05/sam-sutton/hate-crimes-new-york-city-houses-worship/', '/factchecks/2026/may/01/fentrice-driskell/what-does-federal-law-say-about-partisan-gerrymand/', '/factchecks/2026/apr/30/instagram-posts/muro-fronterizo-inmigrantes-eeuu-mexico-ia/', '/factchecks/2026/apr/30/bernie-sanders/one-big-beautiful-bill-ACA-health-care-Medicaid/', '/factchecks/2026/apr/29/tim-scott/secret-service-assassination-shutdown-paychecks/', '/factchecks/2026/apr/29/pine-tree-results-pac/Graham-Platner-Collins-Maine-reddit-rape-Senate/', '/factchecks/2026/apr/29/social-media/correspondents-dinner-shooter-security-footage-AI/', '/factchecks/2026/apr/29/donald-trump/oil-united-states-russia-saudi-arabia-hormuz-iran/', '/factchecks/2026/apr/29/ron-desantis/florida-redistributing-republican-democrat-majority/', '/factchecks/2026/apr/28/social-media/Iran-Trump-tiroteo-cena-casa-blanca/', '/factchecks/2026/apr/23/jay-collins/byron-donalds-florida-military-pay-votes/', '/factchecks/2026/apr/23/robert-f-kennedy-jr/percentage-decreases-drug-prices-trump-math/', '/factchecks/2026/apr/22/joe-rogan/ibogaine-opioid-addiction-psychedelic-withdrawal/', '/factchecks/2026/apr/22/tweets/Biden-Oval-Office-2024-elections/', '/factchecks/2026/apr/22/chris-wright/wind-solar-energy-oil-strait-hormuz-iran/', '/factchecks/2026/apr/21/facebook-posts/Trump-Walter-Reed-AI-video/', '/factchecks/2026/apr/20/democratic-national-committee/trump-defense-budget-ww2/']
```

```
In [168]: final_link=[]
for j in link:
    final="https://www.politifact.com/factchecks/"+j
    final_link.append(final)
print(final_link)
```

```
['https://www.politifact.com/factchecks//factchecks/2026/may/15/tweets/byron-donalds-florida-insider-trading/', 'https://www.politifact.com/factchecks//factchecks/2026/may/14/tweets/hantavirus-quarantine-crisis-actor-rosmarin/', 'https://www.politifact.com/factchecks//factchecks/2026/may/14/kathy-castor/kid-care-florida-desantis-health-insurance/', 'https://www.politifact.com/factchecks//factchecks/2026/may/13/vivek-ramaswamy/Amy-Acton-Ohio-2020-covid-primary-senate/', 'https://www.politifact.com/factchecks//factchecks/2026/may/13/tiktok-posts/hantavirus-infectados-imagenes-crucero/', 'https://www.politifact.com/factchecks//factchecks/2026/may/11/christine-quinn/new-york-city-homeless-children-yankee-stadium/', 'https://www.politifact.com/factchecks//factchecks/2026/may/11/nicholas-langworthy/tax-refunds-11-percent-reduction-irs/', 'https://www.politifact.com/factchecks//factchecks/2026/may/08/donald-trump/gasoline-prices-falling-crude-oil/', 'https://www.politifact.com/factchecks//factchecks/2026/may/08/byron-donalds/florida-voting-rights-act-democrats-gerrymandering/', 'https://www.politifact.com/factchecks//factchecks/2026/may/08/marcy-kaptur/White-House-Trump-ballroom-East-Wing-1-billion/', 'https://www.politifact.com/factchecks//factchecks/2026/may/07/liz-krueger/electricity-data-centers-new-york-households/', 'https://www.politifact.com/factchecks//factchecks/2026/may/07/tweets/california-gasoline-supply-six-weeks/', 'https://www.politifact.com/factchecks//factchecks/2026/may/06/donald-trump/consumer-confidence-sentiment-spending-second-term/', 'https://www.politifact.com/factchecks//factchecks/2026/may/05/sam-sutton/hate-crimes-new-york-city-houses-worship/', 'https://www.politifact.com/factchecks//factchecks/2026/may/01/fentrice-driskell/what-does-federal-law-say-about-partisan-gerrymand/', 'https://www.politifact.com/factchecks//factchecks/2026/apr/30/instagram-posts/muro-fronterizo-inmigrantes-eeuu-mexico-ia/', 'https://www.politifact.com/factchecks//factchecks/2026/apr/30/bernie-sanders/one-big-beautiful-bill-ACA-health-care-Medicaid/', 'https://www.politifact.com/factchecks//factchecks/2026/apr/29/tim-scott/secret-service-assassination-shutdown-paychecks/', 'https://www.politifact.com/factchecks//factchecks/2026/apr/29/pine-tree-results-pac/Graham-Platner-Collins-Maine-reddit-rape-Senate/', 'https://www.politifact.com/factchecks//factchecks/2026/apr/29/social-media/correspondents-dinner-shooter-security-footage-AI/', 'https://www.politifact.com/factchecks//factchecks/2026/apr/29/donald-trump/oil-united-states-russia-saudi-arabia-hormuz-iran/', 'https://www.politifact.com/factchecks//factchecks/2026/apr/29/ron-desantis/florida-redistricting-republican-democrat-majority/', 'https://www.politifact.com/factchecks//factchecks/2026/apr/28/social-media/Iran-Trump-tiroteo-cena-casa-blanca/', 'https://www.politifact.com/factchecks//factchecks/2026/apr/23/jay-collins/byron-donalds-florida-military-pay-votes/', 'https://www.politifact.com/factchecks//factchecks/2026/apr/23/robert-f-kennedy-jr/percentage-decreases-drug-prices-trump-math/', 'https://www.politifact.com/factchecks//factchecks/2026/apr/22/joe-rogan/ibogaine-opioid-addiction-psychedelic-withdrawal/', 'https://www.politifact.com/factchecks//factchecks/2026/apr/22/tweets/Biden-Oval-Office-2024-elections/', 'https://www.politifact.com/factchecks//factchecks/2026/apr/22/chris-wright/wind-solar-energy-oil-strait-hormuz-iran/', 'https://www.politifact.com/factchecks//factchecks/2026/apr/21/facebook-posts/Trump-Walter-Reed-AI-video/', 'https://www.politifact.com/factchecks//factchecks/2026/apr/20/democratic-national-committee/trump-defense-budget-ww2/']
```

```
In [175]: h=soup.find_all('footer',class_="m-statement__footer")
          for i in h:
              print(i.text[-35:-1].strip())
          # for scraping dates
```

```
o • May 15, 2026
o • May 14, 2026
n • May 14, 2026
n • May 13, 2026
o • May 13, 2026
l • May 11, 2026
k • May 11, 2026
on • May 8, 2026
an • May 8, 2026
ño • May 8, 2026
la • May 7, 2026
on • May 7, 2026
on • May 6, 2026
zi • May 5, 2026
an • May 1, 2026
• April 30, 2026
• April 30, 2026
• April 29, 2026
• April 29, 2026
• April 29, 2026
• April 29, 2026
• April 29, 2026
• April 28, 2026
• April 23, 2026
• April 23, 2026
• April 22, 2026
• April 22, 2026
• April 22, 2026
• April 21, 2026
• April 20, 2026
```

```
In [177]: import re
h = soup.find_all('footer', class_="m-statement__footer")

for i in h:

    text = i.get_text(strip=True)

    # Regex for dates like: May 15, 2026
    date = re.search(r'[A-Z][a-z]+ \d{1,2}, \d{4}', text)

    if date:
        print(date.group())
```

```
May 15, 2026
May 14, 2026
May 14, 2026
May 13, 2026
May 13, 2026
May 11, 2026
May 11, 2026
May 8, 2026
May 8, 2026
May 8, 2026
May 7, 2026
May 7, 2026
May 6, 2026
May 5, 2026
May 1, 2026
April 30, 2026
April 30, 2026
April 29, 2026
April 29, 2026
April 29, 2026
April 29, 2026
April 29, 2026
April 28, 2026
April 23, 2026
April 23, 2026
April 22, 2026
April 22, 2026
April 22, 2026
April 21, 2026
April 20, 2026
```

```

In [180]: import requests
from bs4 import BeautifulSoup
import pandas as pd
upperframe=[]
d={'statement':[], 'links':[], 'Date':[], 'source':[]}
for page in range(1,5):
    url='https://www.politifact.com/factchecks/?page='+str(page)
    page1=requests.get(url)
    soup=BeautifulSoup(page1.text, 'html.parser')

    frame=[]

    c=soup.find_all('a',class_="m-statement__name")
    for i in c:
        sources=i.text.strip()
        d['source'].append(sources)
    z=soup.find_all('div',class_='m-statement__quote')
    for i in z:
        statements=i.text.strip()
        d['statement'].append(statements)
    e=soup.find_all('div',class_="m-statement__quote")
    link=[]
    for i in e:
        link.append(i.find('a')['href'])

    for j in link:
        final="https://www.politifact.com"+j
        d['links'].append(final)
    h=soup.find_all('footer',class_="m-statement__footer")
    for i in h:
        dates=i.text[35:-1].strip()
        print(dates)
        d['Date'].append(dates)
A=pd.DataFrame(d)
for i in A['links']:
    print(i)
# making final Links

hormuz-barbary/ (https://www.politifact.com/factchecks/2026/apr/14/byron-donalds-iran-oil-
ckade-trump-strait-hormuz-barbary/)
https://www.politifact.com/factchecks/2026/apr/13/brandon-gill/dignity-act-amnesty-florida-
immigrants-citizenship/ (https://www.politifact.com/factchecks/2026/apr/13/brandon-gill/d
ignity-act-amnesty-florida-immigrants-citizenship/)
https://www.politifact.com/factchecks/2026/apr/13/donald-trump/Pope-Leonuclear-weapon-Ira
n-Trump/ (https://www.politifact.com/factchecks/2026/apr/13/donald-trump/Pope-Leonuclear-
weapon-Iran-Trump/)
https://www.politifact.com/factchecks/2026/apr/13/commonwealth-together-pac/Moulton-Markey
-ICE-resolutions-Trump-Senate-primar/ (https://www.politifact.com/factchecks/2026/apr/13/c
ommonwealth-together-pac/Moulton-Markey-ICE-resolutions-Trump-Senate-primar/)
https://www.politifact.com/factchecks/2026/apr/08/social-media/Moon-photos-artemis-II-AI-N
ASA/ (https://www.politifact.com/factchecks/2026/apr/08/social-media/Moon-photos-artemis-I
I-AI-NASA/)
https://www.politifact.com/factchecks/2026/apr/07/social-media/american-soldier-crying-ira
n-AI/ (https://www.politifact.com/factchecks/2026/apr/07/social-media/american-soldier-cry
ing-iran-AI/)
https://www.politifact.com/factchecks/2026/apr/06/social-media/American-Colonel-pilot-phot
os-AI-Iran/ (https://www.politifact.com/factchecks/2026/apr/06/social-media/American-Colon
el-pilot-photos-AI-Iran/)

```

Static Html

```
In [182]: import bs4
f=open('example.html')
soup = BeautifulSoup(f, 'html.parser')
soup
```

```
Out[182]: <html> <head>
<meta content="text/html;
charset=utf-8" http-equiv="Content-Type"/>
<title>An example of HTML page</title> </head>
<body>
<h2>This is an example HTML page</h2>
<p>
Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nunc at nisi velit, aliquet iaculis
est. Curabitur porttitor nisi vel lacus
eismod egestas. In hac habitasse platea dictumst. In sagittis magna eu odio interdum mollis.
Phasellus sagittis pulvinar facilisis.
Donec vel odio volutpat tortor volutpat commodo. Donec vehicula vulputate sem, vel iaculis ur
na molestie eget. Sed
pellentesque adipiscing tortor, at condimentum elit elementum sed. Mauris dignissim
elementum nunc, non elementum felis condimentum eu. In in turpis quis erat imperdiet vulputat
e. Pellentesque mauris turpis,
dignissim sed iaculis eu, eismod eget ipsum. Vivamus mollis adipiscing viverra. Morbi at sem
eget nisl eismod porta.</p>
<p><a href="https://www.w3resource.com/html/HTML-tutorials.php">Learn HTML from
w3resource.com</a></p>
<p><a href="https://www.w3resource.com/css/CSS-tutorials.php">Learn CSS from
w3resource.com</a></p>
</body>
</html>
```

```
In [183]: print(soup.find('title').text)
```

An example of HTML page

```
In [184]: print(soup.find('h2').text)
```

This is an example HTML page

```
In [185]: print(soup.find('p').text)
```

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nunc at nisi velit, aliquet iaculis est. Curabitur porttitor nisi vel lacus eismod egestas. In hac habitasse platea dictumst. In sagittis magna eu odio interdum mollis. Phasellus sagittis pulvinar facilisis. Donec vel odio volutpat tortor volutpat commodo. Donec vehicula vulputate sem, vel iaculis urna molestie eget. Sed pellentesque adipiscing tortor, at condimentum elit elementum sed. Mauris dignissim elementum nunc, non elementum felis condimentum eu. In in turpis quis erat imperdiet vulputate. Pellentesque mauris turpis, dignissim sed iaculis eu, eismod eget ipsum. Vivamus mollis adipiscing viverra. Morbi at sem eget nisl eismod porta.

```
In [188]: a=soup.find_all('p')
a
```

```
Out[188]: [<p>
  Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nunc at nisi velit, aliquet iaculis
  est. Curabitur porttitor nisi vel lacus
  euismod egestas. In hac habitasse platea dictumst. In sagittis magna eu odio interdum molli
  s. Phasellus sagittis pulvinar facilisis.
  Donec vel odio volutpat tortor volutpat commodo. Donec vehicula vulputate sem, vel iaculis u
  rna molestie eget. Sed
  pellentesque adipiscing tortor, at condimentum elit elementum sed. Mauris dignissim
  elementum nunc, non elementum felis condimentum eu. In in turpis quis erat imperdiet vulputa
  te. Pellentesque mauris turpis,
  dignissim sed iaculis eu, euismod eget ipsum. Vivamus mollis adipiscing viverra. Morbi at se
  m eget nisl euismod porta.</p>,
  <p><a href="https://www.w3resource.com/html/HTML-tutorials.php">Learn HTML from
  w3resource.com</a></p>,
  <p><a href="https://www.w3resource.com/css/CSS-tutorials.php">Learn CSS from
  w3resource.com</a></p>]
```

```
In [190]: for i in a:
  c=i.text
  print(c,'\n')
```

```

  Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nunc at nisi velit, aliquet iaculis
  est. Curabitur porttitor nisi vel lacus
  euismod egestas. In hac habitasse platea dictumst. In sagittis magna eu odio interdum mollis.
  Phasellus sagittis pulvinar facilisis.
  Donec vel odio volutpat tortor volutpat commodo. Donec vehicula vulputate sem, vel iaculis ur
  na molestie eget. Sed
  pellentesque adipiscing tortor, at condimentum elit elementum sed. Mauris dignissim
  elementum nunc, non elementum felis condimentum eu. In in turpis quis erat imperdiet vulputat
  e. Pellentesque mauris turpis,
  dignissim sed iaculis eu, euismod eget ipsum. Vivamus mollis adipiscing viverra. Morbi at sem
  eget nisl euismod porta.
```

```

  Learn HTML from
  w3resource.com
```

```

  Learn CSS from
  w3resource.com
```

```
In [207]: d=soup.find_all('a')
```

```
In [208]: d
```

```
Out[208]: [<a href="https://www.w3resource.com/html/HTML-tutorials.php">Learn HTML from
  w3resource.com</a>,
  <a href="https://www.w3resource.com/css/CSS-tutorials.php">Learn CSS from
  w3resource.com</a>]
```

```
In [209]: for i in d:
  print(i['href'])
```

```

https://www.w3resource.com/html/HTML-tutorials.php (https://www.w3resource.com/html/HTML-tuto
rials.php)
https://www.w3resource.com/css/CSS-tutorials.php (https://www.w3resource.com/css/CSS-tutorial
s.php)
```

```
In [ ]:
```


